K-Nearest Neighbors (KNN)
Lab 8 - Completed Assignment

Dataset: KNN_Project_Data.csv | Records: 1,000 | Features: 10 | Target: TARGET CLASS

This solution follows the order and Python commands in the instructor's KNN lecture: import data, visualize, standardize, split, train and evaluate, test multiple K values, and retrain with K=30. A fixed random_state=42 makes the results reproducible.

1. Import libraries and read the dataset

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

df = pd.read_csv('KNN_Project_Data.csv')
df.head()
```

2. Exploratory data analysis (EDA)

```
sns.pairplot(df, hue='TARGET CLASS')
```

A seaborn pairplot displays feature relationships colored by the target class. The executed figure is included later in this report and in the completed notebook.

3. Standardize the features

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(df.drop('TARGET CLASS', axis=1))
scaled_features = scaler.transform(df.drop('TARGET CLASS', axis=1))
df_feat = pd.DataFrame(scaled_features, columns=df.columns[:-1])
df_feat.head()
```

KNN is distance-based; standardizing the 10 feature columns prevents features with large numeric ranges from dominating the neighbors. TARGET CLASS is excluded from scaling.

4. Train/test split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    scaled_features, df['TARGET CLASS'], test_size=0.30, random_state=42)
```

The split uses 700 training rows and 300 test rows.

5. Initial model: K=1

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

Confusion matrix: [[110, 36], [47, 107]] | Accuracy: 72.33%

```
precision    recall  f1-score   support

           0       0.70      0.75      0.73       146
           1       0.75      0.69      0.72       154

    accuracy                           0.72       300
   macro avg       0.72      0.72      0.72       300
weighted avg       0.73      0.72      0.72       300
```

6. Find a suitable K with the elbow method

```
error_rate = []
for i in range(1, 40):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train, y_train)
    pred_i = knn.predict(X_test)
    error_rate.append(np.mean(pred_i != y_test))

plt.figure(figsize=(10, 6))
plt.plot(range(1, 40), error_rate, color='blue',
         linestyle='dashed', marker='o',
         markerfacecolor='red', markersize=10)
plt.title('Error Rate vs. K Value')
plt.xlabel('K')
plt.ylabel('Error Rate')
```

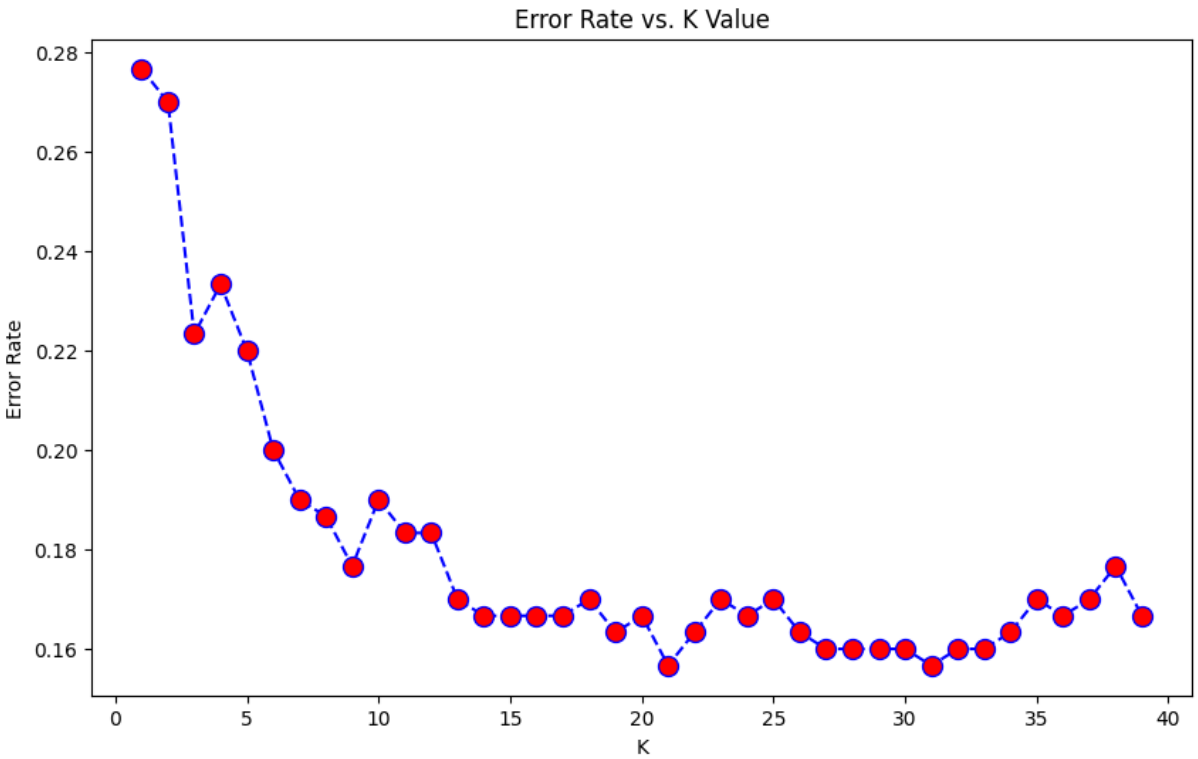


Figure 1. Test-set error rate for K=1 to 39.

7. Retrain the model with K=30

```
knn = KNeighborsClassifier(n_neighbors=30)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)

print('WITH K=30')
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

Confusion matrix: [[127, 19], [29, 125]] | Accuracy: 84.00%

```
precision    recall  f1-score   support

           0       0.81      0.87      0.84       146
           1       0.87      0.81      0.84       154

    accuracy                           0.84       300
   macro avg       0.84      0.84      0.84       300
weighted avg       0.84      0.84      0.84       300
```

8. Results and discussion

Model	Accuracy	Errors in 300 tests
K=1	72.33%	83
K=30	84.00%	48

Increasing the number of neighbors from 1 to 30 improves test accuracy from 72.33% to 84.00%, an increase of 11.67 percentage points. K=30 matches the assignment notebook's illustrated final K value and performs well on this split.

Note: The instructor's example may show different confusion matrices because the original split did not specify a fixed random seed. Also, this notebook follows the lecture by standardizing before splitting; for a production evaluation, the scaler should be fitted on the training data only.

9. EDA figure: pairplot by target class

The pairplot compares all 10 numeric features. The complete image is retained in the executed notebook.

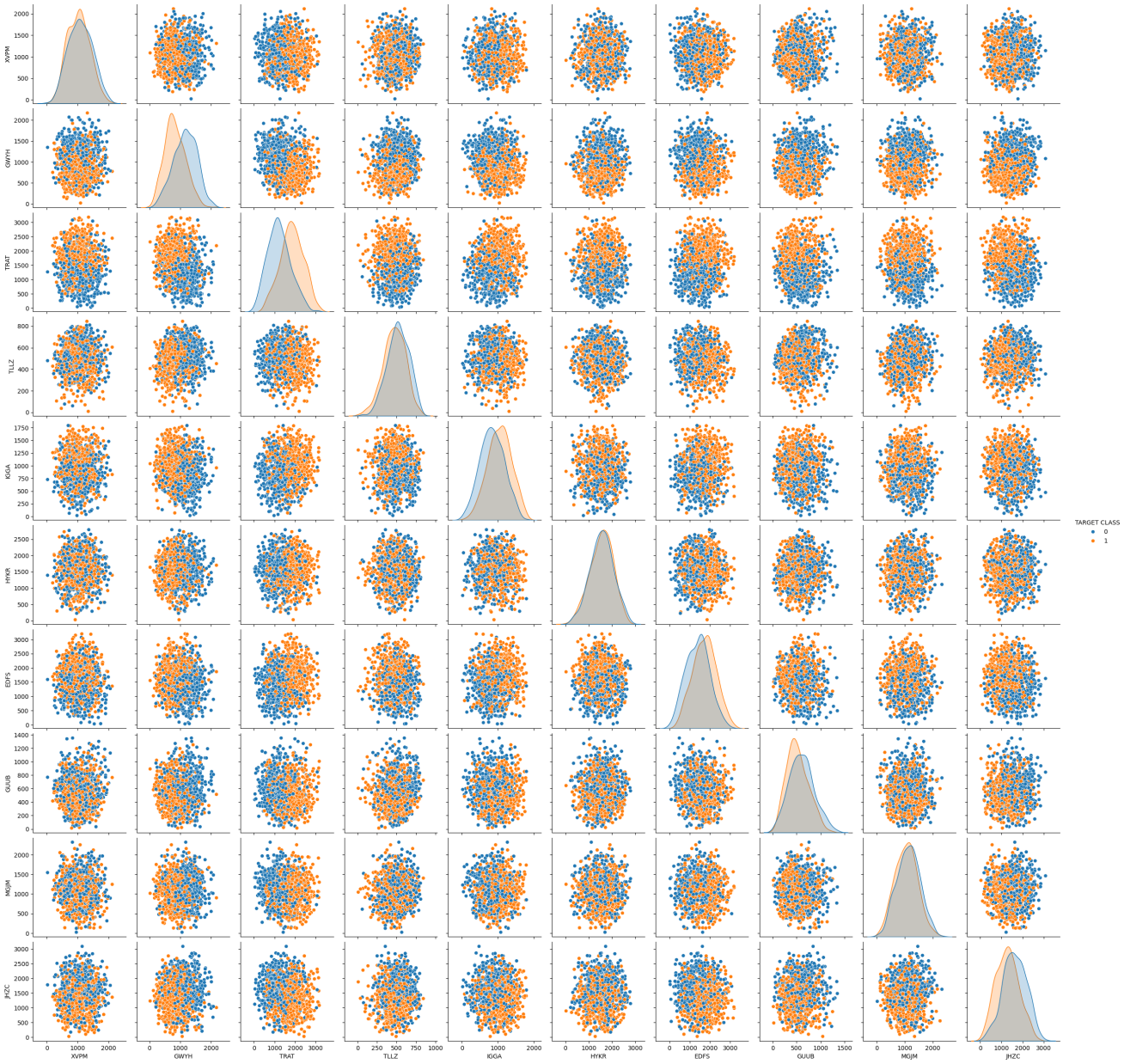


Figure 2. Seaborn pairplot with hue = TARGET CLASS.

Submission files

1. Lab8_KNN_Assignment_Solved.ipynb
2. Lab8_KNN_Solution.py
3. Lab8_KNN_Solution.pdf
4. KNN_Project_Data.csv

The notebook contains the complete code and outputs. The Python script contains the same computation steps in a regular .py format. The original dataset was supplied without an extension, so a .csv copy is included and the data-reading line was updated to match.